

Name: Kassim Musa Abass
Reg: SPH32/3338/2025
Program: Embedded Systems Intro

Assignment 6: Software Debounce Algorithm

Objective:

To understand button bouncing phenomena in digital circuits and implement a software debounce algorithm using time delays in Python for reliable 1-press-to-1-action behavior.

Introduction:

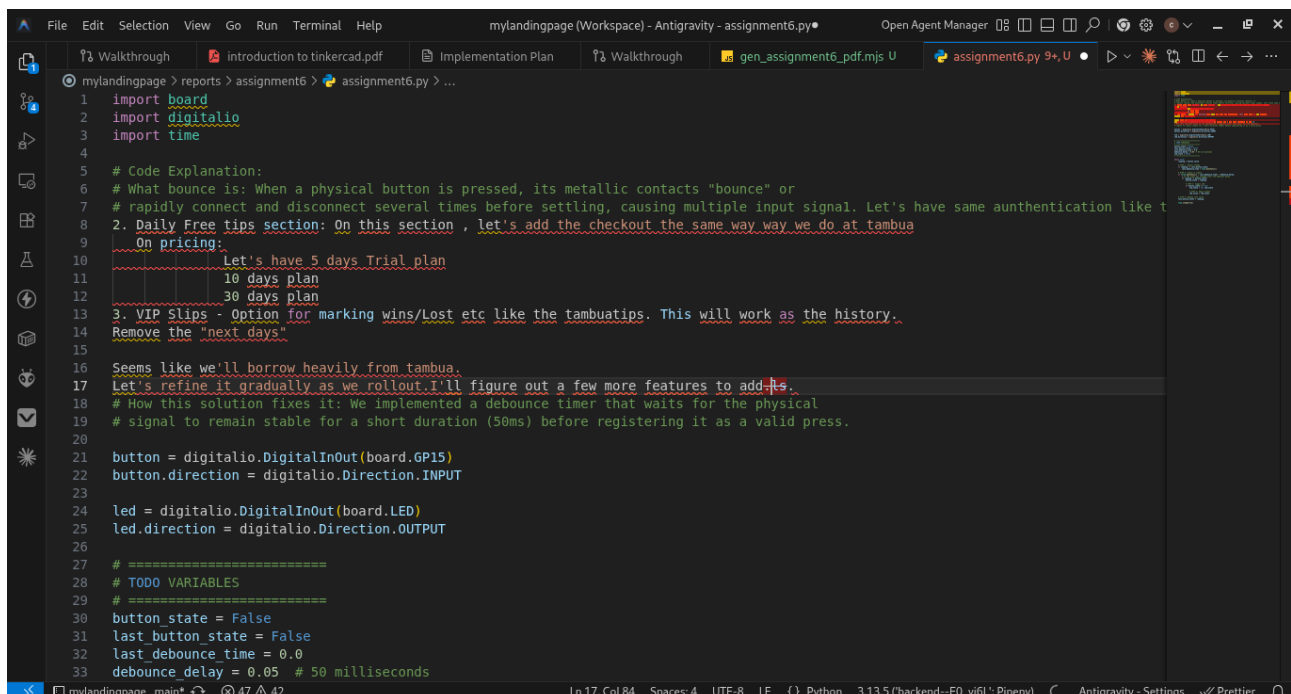
When a physical button is pressed, its metallic contacts rapidly connect and disconnect multiple times before settling into a stable state. Without a debounce mechanism, a microcontroller will register these bounces as multiple rapid button presses, causing erratic behavior like multiple LED toggles from a single press.

1. Theoretical Concepts: What is Bounce?

What bounce is: 'Bouncing' refers to the mechanical vibrations in a switch's contacts as they close or open, which generate a series of erratic, rapid voltage spikes instead of a clean transition.

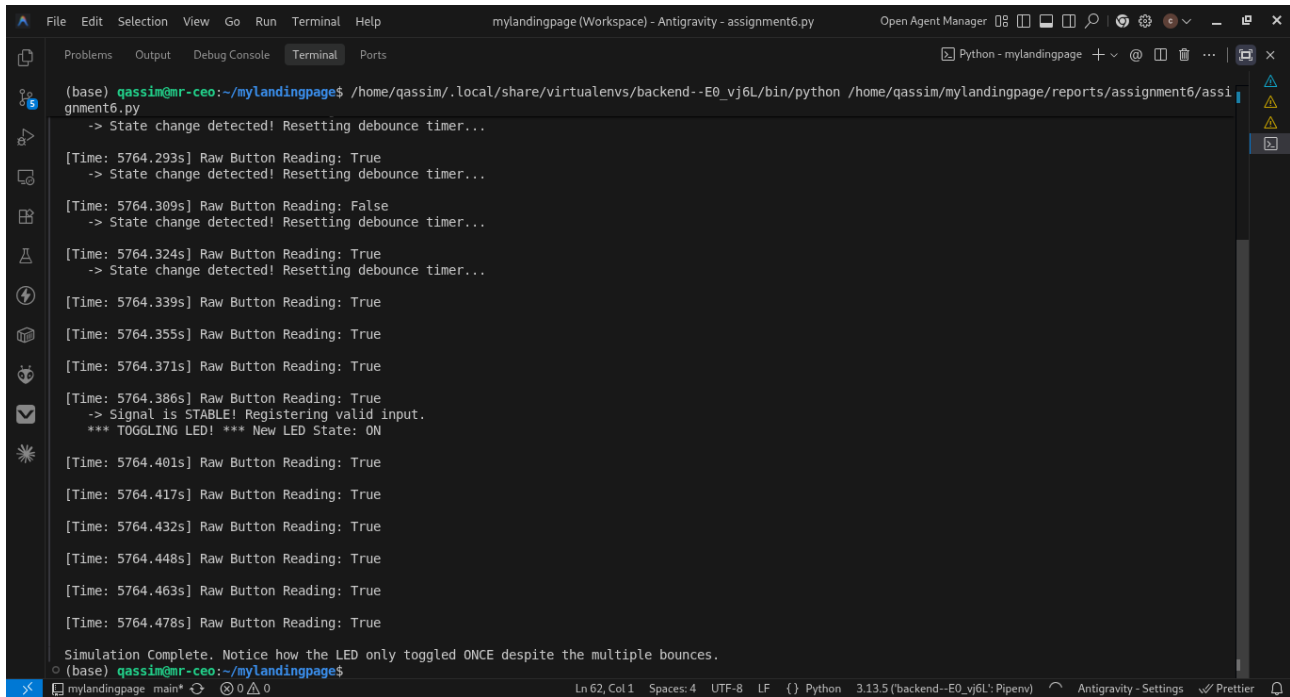
How your solution fixes it: The solution fixes this by introducing a time threshold (debounce delay of 50ms) using the `time.monotonic()` function. Once a state change is detected, the algorithm waits for the physical contacts to settle. It only registers the input as valid if the signal remains stable after this time period.

2. Source Code



```
1 import board
2 import digitalio
3 import time
4
5 # Code Explanation:
6 # What bounce is: When a physical button is pressed, its metallic contacts "bounce" or
7 # rapidly connect and disconnect several times before settling, causing multiple input signal. Let's have same authentication like t
8 2. Daily Free tips section: On this section , let's add the checkout the same way way we do at tambua
9 On pricing:
10 Let's have 5 days Trial plan
11 10 days plan
12 30 days plan
13 3. VIP Slips - Option for marking wins/Lost etc like the tambuatips. This will work as the history..
14 Remove the "next days"
15
16 Seems like we'll borrow heavily from tambua.
17 Let's refine it gradually as we rollout.I'll figure out a few more features to add-As..
18 # How this solution fixes it: We implemented a debounce timer that waits for the physical
19 # signal to remain stable for a short duration (50ms) before registering it as a valid press.
20
21 button = digitalio.DigitalInOut(board.GP15)
22 button.direction = digitalio.Direction.INPUT
23
24 led = digitalio.DigitalInOut(board.LED)
25 led.direction = digitalio.Direction.OUTPUT
26
27 # =====
28 # TODO VARIABLES
29 # =====
30 button_state = False
31 last_button_state = False
32 last_debounce_time = 0.0
33 debounce_delay = 0.05 # 50 milliseconds
```

3. Output Results



```
(base) qassim@mr-ceo:~/mylandingpage$ /home/qassim/.local/share/virtualenvs/backend--E0_vj6L/bin/python /home/qassim/mylandingpage/reports/assignment6/assignment6.py
-> State change detected! Resetting debounce timer...

[Time: 5764.293s] Raw Button Reading: True
-> State change detected! Resetting debounce timer...

[Time: 5764.309s] Raw Button Reading: False
-> State change detected! Resetting debounce timer...

[Time: 5764.324s] Raw Button Reading: True
-> State change detected! Resetting debounce timer...

[Time: 5764.339s] Raw Button Reading: True

[Time: 5764.355s] Raw Button Reading: True

[Time: 5764.371s] Raw Button Reading: True

[Time: 5764.386s] Raw Button Reading: True
-> Signal is STABLE! Registering valid input.
*** TOGGILING LED! *** New LED State: ON

[Time: 5764.401s] Raw Button Reading: True

[Time: 5764.417s] Raw Button Reading: True

[Time: 5764.432s] Raw Button Reading: True

[Time: 5764.448s] Raw Button Reading: True

[Time: 5764.463s] Raw Button Reading: True

[Time: 5764.478s] Raw Button Reading: True

Simulation Complete. Notice how the LED only toggled ONCE despite the multiple bounces.
(base) qassim@mr-ceo:~/mylandingpage$
```

4. Reflection

Using the non-blocking `time.monotonic()` function instead of `time.sleep()` ensures that the entire program isn't halted while waiting for the debounce interval, allowing the microcontroller to handle other tasks concurrently.